hdoc day 33 by trishil chauhan

Aim

To write a menu-driven C program using separate functions and variable-length 2D integer arrays to verify the associative properties of matrix addition and matrix multiplication.

Theory

Addition: For matrices A, B and C of identical dimensions m x n, (A+B)+C = A+(B+C).
Multiplication: For A (m x n), B (n x p) and C (p x q), (A x B) x C = A x (B x C). Both results have dimensions m x q.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void printMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++)
            printf("%6d", a[i][j]);
        printf("\n");
    }
}

void add(int r, int c, int a[r][c], int b[r][c],
         int result[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            result[i][j] = a[i][j] + b[i][j];
}

void multiply(int r, int common, int c,
              int a[r][common], int b[common][c],
              int result[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) {
            result[i][j] = 0;
            for (int k = 0; k < common; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
}

int equal(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void verifyAddition(void) {
    int m, n;
    printf("Enter rows and columns (m n): ");
    if (scanf("%d%d", &m, &n) != 2 || m <= 0 || n <= 0) {
        printf("Invalid dimensions.\n"); return;
    }
    int A[m][n], B[m][n], C[m][n];
```

```
    int AB[m][n], BC[m][n], left[m][n], right[m][n];
    printf("Enter A (%d elements):\n", m*n);
    readMatrix(m, n, A);
    printf("Enter B (%d elements):\n", m*n);
    readMatrix(m, n, B);
    printf("Enter C (%d elements):\n", m*n);
    readMatrix(m, n, C);
    add(m, n, A, B, AB);
    add(m, n, AB, C, left);
    add(m, n, B, C, BC);
    add(m, n, A, BC, right);
    printf("(A+B)+C:\n"); printMatrix(m, n, left);
    printf("A+(B+C):\n"); printMatrix(m, n, right);
    printf("Addition associative: %s\n",
           equal(m, n, left, right) ? "YES" : "NO");
}

void verifyMultiplication(void) {
    int m, n, p, q;
    printf("Enter m n p q: ");
    if (scanf("%d%d%d%d", &m, &n, &p, &q) != 4 ||
        m <= 0 || n <= 0 || p <= 0 || q <= 0) {
        printf("Invalid dimensions.\n"); return;
    }
    int A[m][n], B[n][p], C[p][q];
    int AB[m][p], BC[n][q], left[m][q], right[m][q];
    printf("Enter A (%d x %d):\n", m, n);
    readMatrix(m, n, A);
    printf("Enter B (%d x %d):\n", n, p);
    readMatrix(n, p, B);
    printf("Enter C (%d x %d):\n", p, q);
    readMatrix(p, q, C);
    multiply(m, n, p, A, B, AB);
    multiply(m, p, q, AB, C, left);
    multiply(n, p, q, B, C, BC);
    multiply(m, n, q, A, BC, right);
    printf("(A*B)*C:\n"); printMatrix(m, q, left);
    printf("A*(B*C):\n"); printMatrix(m, q, right);
    printf("Multiplication associative: %s\n",
           equal(m, q, left, right) ? "YES" : "NO");
}

int main(void) {
    int choice;
    do {
        printf("\n1. Verify addition associativity\n");
        printf("2. Verify multiplication associativity\n");
        printf("3. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) break;
        switch (choice) {
            case 1: verifyAddition(); break;
            case 2: verifyMultiplication(); break;
            case 3: printf("Exiting...\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while (choice != 3);
    return 0;
}
```

Sample Run

```
Choice: 1; m = 1, n = 2
A: 1 2     B: 3 4     C: 5 6
(A+B)+C:  9  12
A+(B+C):  9  12
Addition associative: YES

Choice: 2; m = n = p = q = 2
```

```
A: 1 2 / 3 4    B: 2 0 / 1 2    C: 1 1 / 0 1
(A*B)*C:   4  8 / 10 18
A*(B*C):   4  8 / 10 18
Multiplication associative: YES
```

Result

The program calculates both parenthesizations and compares their corresponding elements. For valid integer matrices, both associative properties are verified (subject to avoiding integer overflow).